

GAMES & PROBLEMS

Contents

1. 8 Puzzle Problem.....	1
2. Vacuum Cleaner problem.....	2
3. Water Jug Problem	3
4. Missionaries and Cannibals Problem.....	4
5. N-Queens Problem	4
6. Tower of Hanoi.....	5
7. Tic-Tac-Toe	6
8. Cryptarithmic Problem.....	6
9. Minimax Algorithm	8
10. Alpha-Beta Pruning.....	9

8 Puzzle Problem

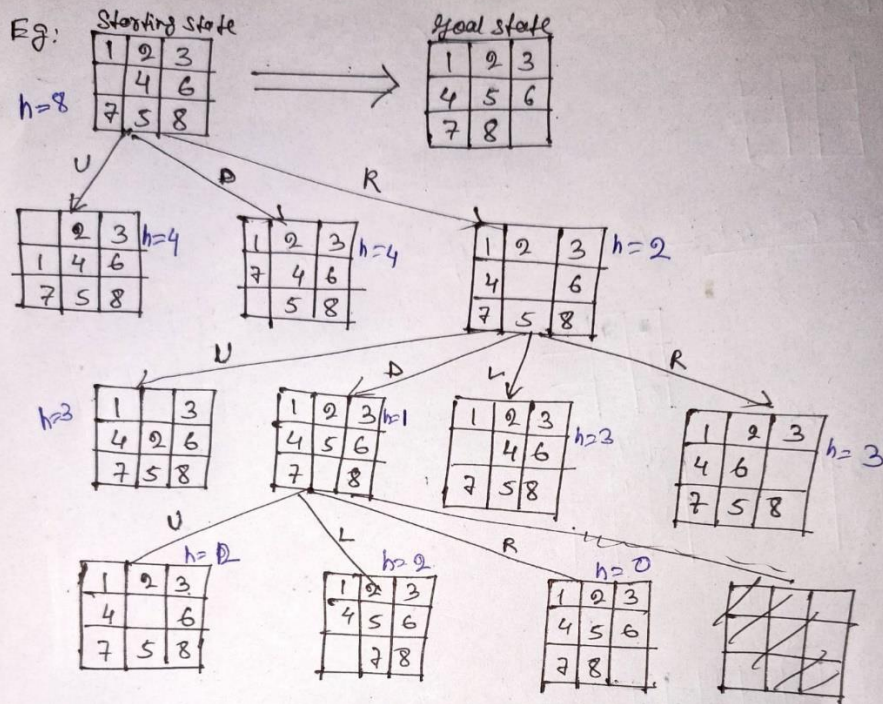
The 8-puzzle consists of a 3x3 grid with 8 numbered tiles and one empty space. The goal is to move the tiles around by sliding them into the empty space to achieve a specified goal state (usually the tiles ordered from 1 to 8).

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions:** The simplest formulation defines the actions as movements of the blank space *Left*, *Right*, *Up*, or *Down*. Different subsets of these are possible depending on where the blank is.
- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left* to the start state in Figure 3.4, the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

- **Constraints:**
 - Not all moves are possible in every state, as some tiles are on the boundary of the grid.
 - The number of inversions determines if the puzzle is solvable (even numbers of inversions imply solvability).
- **Search Techniques:** Often solved with A* search, **Breadth-First Search (BFS)**, or **Depth-First Search (DFS)**.

8-puzzle problem with heuristic

Here we assume the heuristic is "no of miss. placed tiles from goal state."



Vacuum Cleaner problem

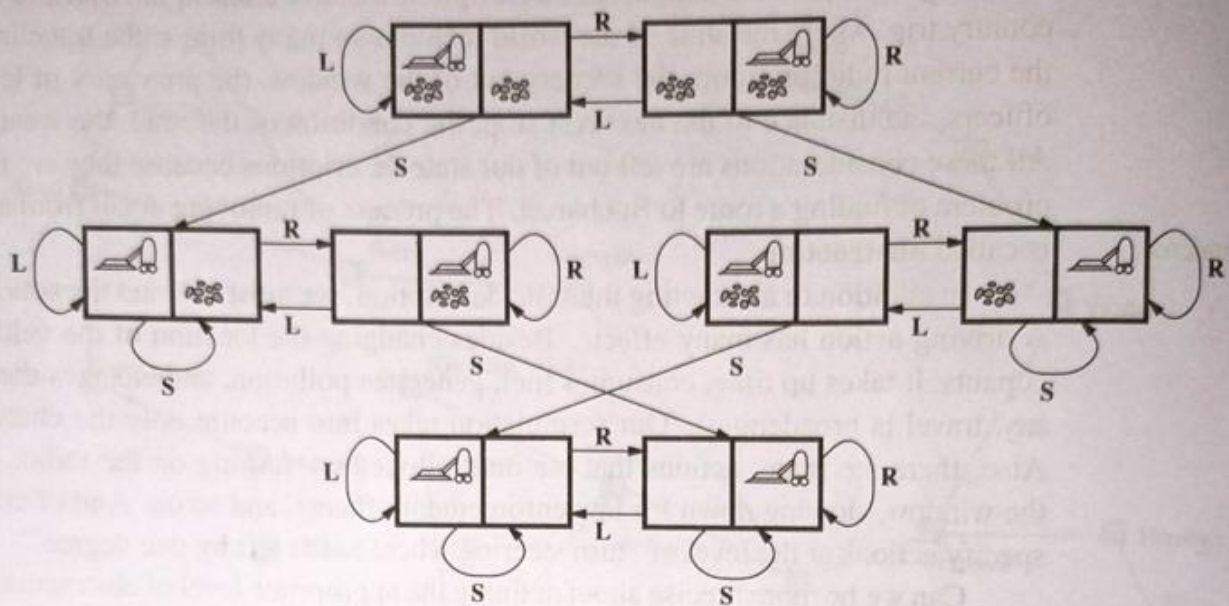


Figure 3.3 The state space for the vacuum world. Links denote actions: L = Left, R = Right, S = Suck.

3.2.1 Toy problems

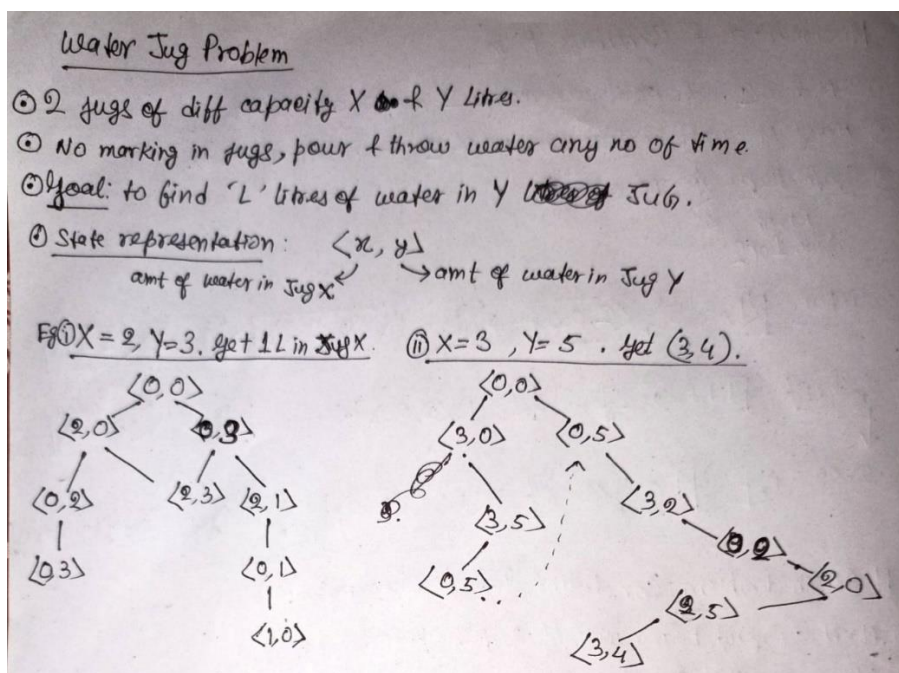
The first example we examine is the **vacuum world** first introduced in Chapter 2. (See Figure 2.2.) This can be formulated as a problem as follows:

- **States:** The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt. Thus, there are $2 \times 2^2 = 8$ possible world states. A larger environment with n locations has $n \cdot 2^n$ states.
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** In this simple environment, each state has just three actions: *Left*, *Right*, and *Suck*. Larger environments might also include *Up* and *Down*.
- **Transition model:** The actions have their expected effects, except that moving *Left* in the leftmost square, moving *Right* in the rightmost square, and *Sucking* in a clean square have no effect. The complete state space is shown in Figure 3.3.
- **Goal test:** This checks whether all the squares are clean.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

Water Jug Problem

The problem involves two jugs with different capacities and an infinite water supply. The goal is to measure out an exact quantity of water using the two jugs.:

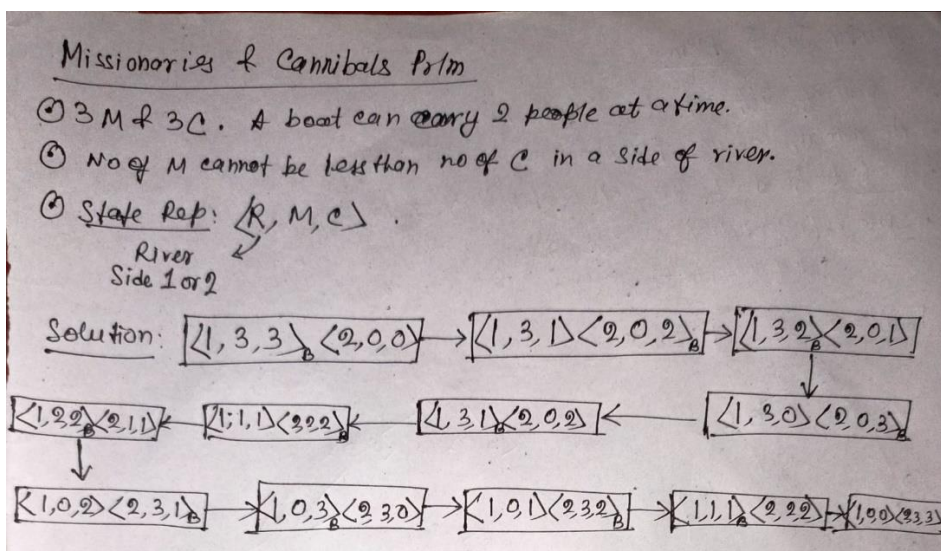
- **State space:** A state is represented as a pair of numbers (x, y) , where x is the amount of water in jug 1, and y is the amount in jug 2.
- **Initial state:** Both jugs are empty $(0, 0)$.
- **Goal state:** The target amount of water in either one of the jugs (e.g., 4 liters in one of the jugs).
- **Actions:** Fill a jug completely, empty a jug., Pour water from one jug to another until one jug is full, or the other is empty.
- **Constraints:**
 - The capacity of each jug is fixed.
 - You cannot measure intermediate amounts without performing valid actions.
- **Search Techniques:** BFS, DFS, or A* are common approaches for solving the problem



Missionaries and Cannibals Problem

The problem involves 3 missionaries and 3 cannibals who need to cross a river using a boat. The challenge is to ensure that at no point do the cannibals outnumber the missionaries on either side of the river (as this would lead to the missionaries being eaten).

- **State space:** A state can be represented as a tuple $(M_left, C_left, boat)$, where M_left and C_left are the number of missionaries and cannibals on the left side of the river, and the boat indicates its position (left or right).
- **Initial state:** All 3 missionaries and 3 cannibals are on the left bank, with the boat on the left.
- **Goal state:** All missionaries and cannibals are on the right bank.
- **Actions:** Move 1 or 2 people across the river in the boat (combinations of missionaries and cannibals).
- **Constraints:**
 - The boat can only hold up to 2 people.
 - You cannot have more cannibals than missionaries on either side of the river (except when there are no missionaries).
- **Search Techniques:** The problem can be solved using **BFS**, **DFS**, or **Iterative Deepening Search (IDS)**.

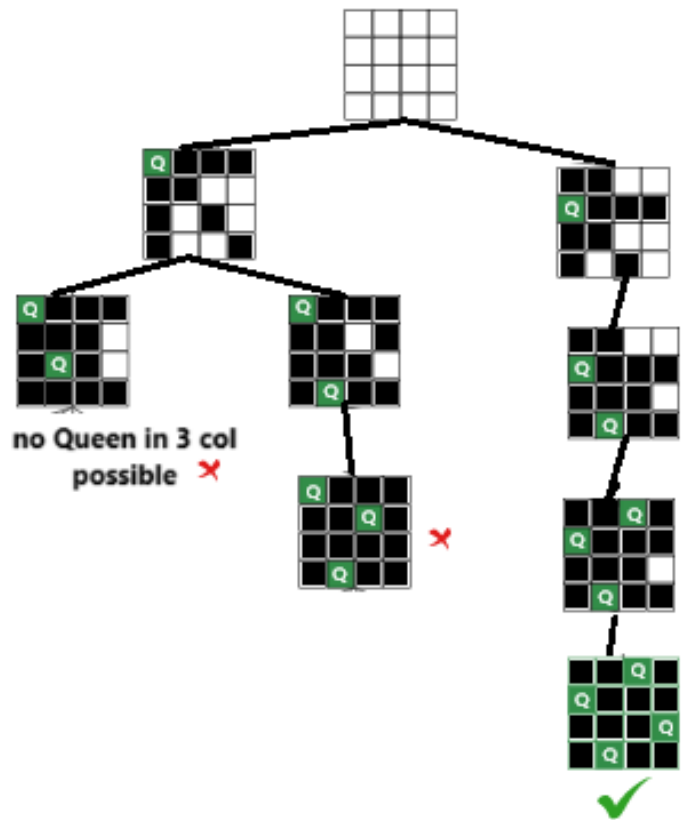
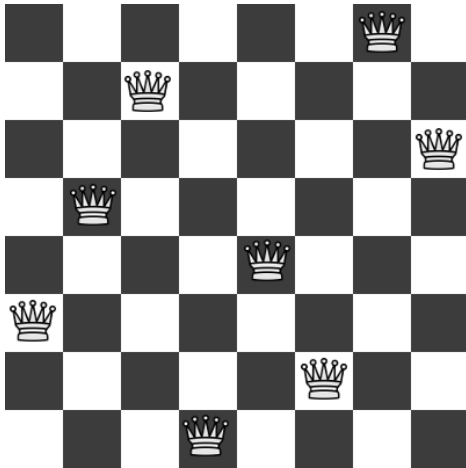


N-Queens Problem

The objective is to place N queens on an $N \times N$ chessboard such that no two queens threaten each other (i.e., no two queens share the same row, column, or diagonal).

- **State space:** Every arrangement of queens on the board represents a state.
- **Initial state:** An empty board.
- **Goal state:** A valid arrangement of N queens where no two queens are attacking each other.
- **Actions:** Place a queen in an available, non-attacking position on the board.
- **Constraints:** No two queens can be in the same row, column, or diagonal.
- **Number of Solutions:**
 - 4 Queens: There are 2 solutions.
 - 8 Queens: There are 92 solutions.

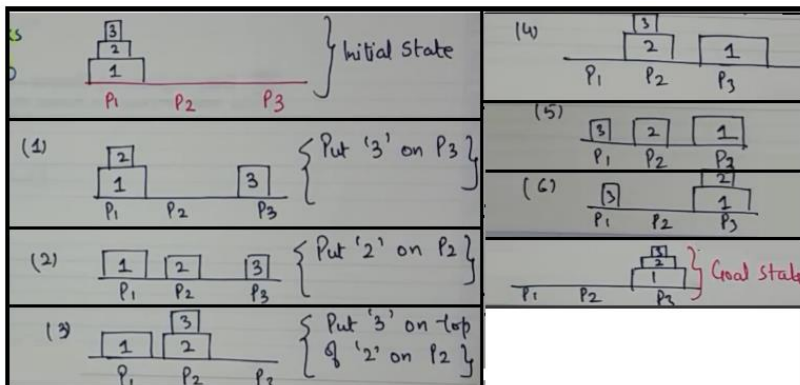
- **Search Techniques:** Solved using Backtracking, Constraint Satisfaction, or Local Search (Hill Climbing, Simulated Annealing).



Tower of Hanoi

The problem involves 3 pegs and N disks of different sizes. The objective is to move all disks from the first peg to the third peg, following these rules:

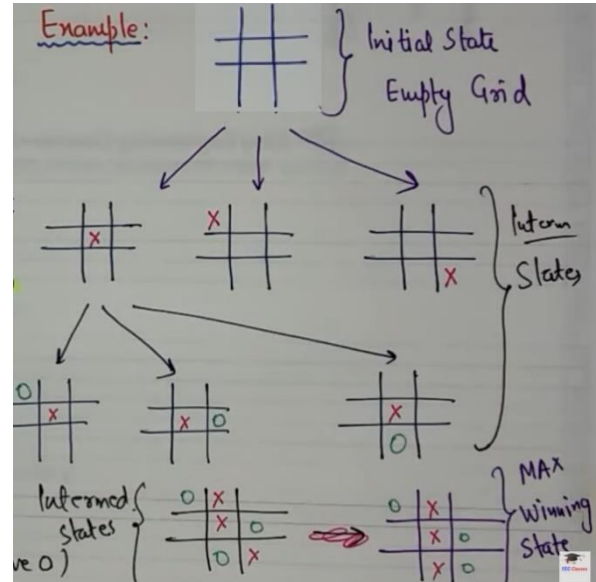
- Only one disk can be moved at a time.
- A disk can only be placed on top of a larger disk.
- You can use the second peg as an auxiliary.
- **State space:** Every arrangement of the disks on the three pegs represents a state.
- **Initial state:** All disks are on the first peg.
- **Goal state:** All disks are on the third peg, in the same order.
- **Actions:** Move a disk from one peg to another, subject to the rules.
- **Constraints:**
 - Only one disk can be moved at a time.
 - A larger disk cannot be placed on a smaller disk.
- **Number of Moves:**
 - For N disks, the minimum number of moves required is $2^N - 1$.
 - Example for 3 disks: **7 moves**.
- **Search Techniques:** Solved using **recursion** or **dynamic programming**.



Tic-Tac-Toe

A two-player game played on a 3x3 grid. The players take turns marking a space on the grid with their symbol (either X or O). The goal is to get three of their symbols in a row (either horizontally, vertically, or diagonally).

- **State space:** Every arrangement of Xs and Os on the board represents a state.
- **Initial state:** An empty 3x3 board.
- **Goal state:** A configuration where either player has aligned 3 of their symbols in a row, or a draw if no moves are left.
- **Actions:** Place X or O in an empty square.
- **Constraints:**
 - Players alternate turns.
 - A player can only place their symbol in an empty square.
 - The game ends when a player wins or there are no more moves (draw).
- **Search Techniques:** Solved using **Minimax Algorithm** (with or without alpha-beta pruning), which explores all possible game states to find an optimal move for the current player.



Cryptarithmic Problem

A **cryptarithmic** problem is a mathematical puzzle where the digits in a given arithmetic equation are replaced by letters (or symbols). The objective is to assign digits to the letters such that the arithmetic equation holds true. Each letter represents a unique digit.

- **Goal:** Here, each letter represents a unique digit, and the goal is to find the digit assignments such that the equation is valid.
- **State space:** Each possible assignment of digits to letters is a state.
- **Initial state:** No letters are assigned digits (all letters are variables).
- **Goal state:** A valid digit assignment where the arithmetic equation is satisfied.
- **Actions:** Assign a digit (0-9) to a letter, ensuring no two letters are assigned the same digit. Once all letters are assigned, verify whether the arithmetic equation holds true.
- **Constraints:**
 - Each letter must represent a unique digit.
 - No two letters can have the same digit.
 - The arithmetic equation must be correct after the assignment of digits.
 - Often, the leftmost letter of a number cannot be zero (e.g., in "SEND", 'S' cannot be 0).
- **Search Techniques:**
 - **Backtracking:** A common approach to solving cryptarithmic problems, where we try assigning digits to letters one by one, backtracking if we find a conflict or if the arithmetic equation is not satisfied.
 - **Constraint Satisfaction Problem (CSP):** The problem can be formulated as a CSP where the variables are the letters, the domain is the set of digits, and the constraints are based on the uniqueness of the digits and the correctness of the arithmetic operation.

Cryptarithmic Problem

Q.1)
$$\begin{array}{r} T O \\ G O \\ \hline O U T \end{array}$$
 Ans) Left most value of Answer that is 0 is always 1 (due to carry). $\boxed{O=1}$

$$\begin{array}{r} T 1 \\ G 1 \\ \hline 1 U T \end{array} \rightarrow \begin{array}{r} 2 1 \\ G 1 \\ \hline 1 U 2 \end{array} \quad \therefore \boxed{T=2}$$
 we see. $2+G=U$

Now $T+G$ must be >9 so that we get carry for $O=1$.

Let $\boxed{G=8}$
$$\begin{array}{r} 2 1 \\ 8 1 \\ \hline 1 0 2 \end{array}$$
 $\therefore \boxed{U=0}$

$$\begin{array}{l} T=2 \\ O=1 \\ U=0 \\ G=8 \end{array}$$

Q.2)
$$\begin{array}{r} S E N D \\ M O R E \\ \hline M O N E Y \end{array} \rightarrow \begin{array}{r} S E N D \\ M O R E \\ \hline M O N E Y \end{array}$$
 , Now $S+1 > 10$. Assume $\boxed{S=9}$, then $\boxed{O=0}$
 $\boxed{M=1}$

$$\begin{array}{r} 9 E N D \\ 1 0 R E \\ \hline 1 1 0 N E Y \end{array}$$
 , As $E \neq N$, So there should be a carry from $(N+R)$
 Let $\boxed{E=4}$, $N \Rightarrow E+O+1=6 \Rightarrow \boxed{N=6}$

$$\begin{array}{r} 9 5 6 D \\ 1 0 R 5 \\ \hline 1 0 6 5 Y \end{array}$$
 , $6+R=5$ with a carry 1
 $\therefore R=9$. X this is wrong as $S=9$ already.
 So there must be a carry from $(D+5)$

$\therefore 6+1+R=15$, Now. $D+5=Y$ with a carry.
 $\boxed{R=8}$
 Assume $\boxed{D=7}$ ($N=6$ & $E=5$ so we can't take)
 $\boxed{Y=2}$

Q.3)
$$\begin{array}{r} E A T \\ T H A T \\ \hline A P P L E \end{array}$$

$$\begin{array}{r} 8 1 9 \\ 9 2 1 9 \\ \hline 1 0 0 3 8 \end{array}$$
 $\boxed{A=1} \rightarrow \boxed{L=2} (A+L)$
 $\boxed{T=9} \rightarrow \boxed{P=0} (T+P=10)$
 $E=T+T=18 \Rightarrow \boxed{E=8}$
 $H+E=P$
 $\Rightarrow H+8=0 \Rightarrow \boxed{H=2}$

Q.4) SOME TIME SPENT

1	9	3	4	
8	5	3	4	
1	0	4	6	8

$S = 1$
 $T = 2E, \therefore T = \text{even}$
 $N = 2M, \therefore N = \text{even}$

Assume $T = 8 \rightarrow E = 4$ | $P = S + T = 1 + 8 = 9 \Rightarrow P = 9$ (1 for carry)

Assume $N = 6 \rightarrow M = 3$ | $O + I = 4$

Let $D = 9, I = 5$

Q.5) BASE BALL GAMES

7	4	7	2	
7	4	5	5	
1	4	9	2	7

$G = 1$

We see $E + L = 8, S + L = E$

Assume $E + L = 8$ with carry $\Rightarrow E + L = S + 10$

$\Rightarrow E = S - L + 10$ | $\therefore S - E = 5$

$\Rightarrow S + L = S - L + 10$ | $(S, E) \in \{(5, 0), (6, 1), (7, 2), (8, 3), (9, 4)\}$

$\Rightarrow L = 5$ | $\times$ $\times$

Now, $B + B + \text{carry} = A + \text{carry} \rightarrow B$ can be 5, 6, 7, 8, 9 (as $L = 5$)

Case 1: Let $(S, E) = (7, 2), S = 7, E = 2, B = 6$

$\therefore A = B + B = 6 + 6 = 12 \Rightarrow A = 2$ X NOT possible

Case 2: Let $S = 8, E = 3, B = 7$

$A = 7 + 7 = 14 \Rightarrow A = 4$ | $M = 4 + 4 = 8 \Rightarrow M = 8 + 1 \Rightarrow M = 9$

Minimax Algorithm

The **Minimax Algorithm** is a decision-making algorithm used in **two-player zero-sum games**, where one player's gain is the other player's loss. It's commonly used in games like **Tic-Tac-Toe**, **Chess**, and **Checkers**. The algorithm assumes that both players play optimally. **Key Concepts:**

- **Maximiser:** One player tries to **maximize** their score.
- **Minimizer:** The opponent tries to **minimize** the maximiser's score.
- **Game Tree:** Represents all possible moves for both players. Each node in the tree is a state of the game.
- Solution is found if it exists.

Example of Alpha-Beta Pruning:

1. **Max** is trying to find the best move.
2. **Min** is evaluating a branch, but finds that its value is less than **alpha**, meaning the Max player will never allow it to happen.
3. The algorithm stops evaluating the remaining branches (prunes them) because they cannot lead to a better outcome for Min.

